CIS613 - W26 - Half Semester Project
Testing Java Atomic Classes

Ahsanul Kabir

May 8, 2026

Contents

1 Introduction **3**

2 Premise **3**

3 Test Plan **3**

4 Test Generation **3**
4.1 Process . . . 3
4.2 Rationale . . . 4

5 Test Execution **4**
5.1 Testing AtomicBoolean . . . 4
5.1.1 Testing Base Class and Primitive . . . 4
5.1.2 Testing Functionality and Atomicity . . . 4
5.2 Testing AtomicInteger . . . 6
5.2.1 Testing Base Class and Primitive . . . 6
5.2.2 Testing Functionality and Atomicity . . . 6

6 Testing Outcome **10**
6.1 AtomicBooleanTest . . . 10
6.1.1 Stale Condition with Primitive and Base Class . . . 10
6.1.2 Functionality Test . . . 11
6.1.3 Atomicity with Atomic Class . . . 11
6.2 AtomicIntegerTest . . . 12
6.2.1 Race Condition with Primitive and Base Class . . . 12
6.2.2 Functionality Test . . . 13
6.2.3 Atomicity with Atomic Class . . . 13

7 Test Metrics **15**
7.1 Test Coverage . . . 15
7.2 Mutation Testing . . . 15

8 Model Checking **16**

9 Discussion **18**

Appendices **19**

A AI Usage **19**

1 Introduction

I wanted to write test code for the Java utility library. I needed to narrow down my scope as the Java library is vast. I decided to write test code for $AtomicBoolean$ and $AtomicInteger$ classes defined under the *concurrent* package. My primary motivation here is to find out why the Java library defined extra classes for thread safety despite having the base classes and primitive types.

2 Premise

- Language: Java 17
- Version: 17.0.18
- Distribution: Amazon Corretto
- IDE: IntelliJ
- Maven: Apache Maven 3.9.14
- GitHub: github.com/kowshid/cis-613-project

I created a new project in IntelliJ and copied the $AtomicBoolean$ and $AtomicInteger$ classes from the library into my *src* directory under *util.concurrent.atomic* package. I created the test classes within the same package later. I only kept methods related to eager evaluation, as lazy evaluation is outside the scope of this project.

3 Test Plan

I planned the test cases in three steps:

1. Test how base $Boolean$ and $Integer$ classes and primitives behave in multi-threaded scenario. This can establish the need for *atomic* classes.
2. Test functionality of $AtomicBoolean$ and $AtomicInteger$. This is to verify that the methods in these classes work as expected in the atomic classes.
3. Test Atomicity of the Atomic classes. This will demonstrate that the atomic classes overcome the non thread-safety of the base classes.

4 Test Generation

4.1 Process

First, I created test cases that demonstrate non thread safety for $Boolean$ class primitive $boolean$ using two threads. Then race conditions are created for $Integer$ classes and *int* is generated using multiple threads starting at the same time and reading and writing the same variable.

For $AtomicBoolean$ class, I followed Equivalence Class Testing for the inputs. As the possible classes are just $true$ and $false$ in this case, the test suite does not grow exponentially and remains manageable without compromising the quality.

For $AtomicInteger$ class, I tried to follow Robust Boundary Value Testing for the single input methods. For some methods, I only did special case testing. At the end, the test suite contained some Normal Boundary Value Testing as well. For robust boundary value testing, I used values $INTEGER_MIN - 1$, $INTEGER_MIN$, -1, 0, 1, $INTEGER_MAX$, $INTEGER_MAX + 1$.

To test the atomiticity, I used similar approach for both atomic classes. All the methods are tested for one variable declared with atomicity, using multiple threads starting at the same time and reading and writing the same variable. To be sure about the results, I repeated the testing 20 times for all the thread safety tests. Thread count in all cases was

999. I picked these numbers arbitrarily. Further tests can be done to determine sufficient parameters to identify errors, which is not the focus of this project.

The assumption for atomicity related codes is *CountDownLatch* works as intended, making the thread start at the same time. This is used as a synchronizer for the threads. Another common pattern I used is declaration of an array with a single element for threading related codes. I needed the variable wrapper as variables used inside a lambda must be final or effectively final. Just declaring an *Integer* or *int* or *AtomicInteger* would prevent the value from being updated inside executor service.

4.2 Rationale

I wrote the first phase to demonstrate that non-thread safety is inherent in the design of the non-atomic types in Java. They are not designed based on how variables would behave in a high-concurrent situation, but rather to manage the normal non-concurrent scenarios more efficiently. Any evidence of stale or race conditions using base classes and primitive types is enough to necessitate separate atomic classes where necessary.

The second phase is to verify that atomic classes work as intended with boundary value testing and equivalent class testing. Before being able to handle high concurrency or avoid race conditions, the methods used should perform their basic task correctly. I designed the test suite as exhaustively as possible to make sure the code works fine under normal non-concurrent conditions.

The third part of the tests works with a common atomic class instance, working as intended when a lot of threads work on that shared variable. All the tests follow the same pattern. When one shared variable is updated, all the other threads should not see any stale values from the past. The update should be available to all the threads starting together, and threads should be able to execute code based on the updates. The logic here demonstrates that for a project with high concurrency, multiple threads, atomic classes do not behave randomly, unlike the other types. I repeated all the concurrency tests 20 times to demonstrate that the probability of non-atomic behavior for atomic classes is close to zero.

5 Test Execution

5.1 Testing AtomicBoolean

5.1.1 Testing Base Class and Primitive

- Test Class: *BaseClassAndPrimitiveConcurrencyTest*
- Methods:
  - testPrimitiveBooleanStaleCondition
  - testBooleanStaleCondition
  - testAtomicBooleanNonStaleCondition

5.1.2 Testing Functionality and Atomicity

- Test Class: *AtomicBooleanTest*
- Test Methods:
  1. testDefaultConstructor
  2. testConstructWithTrue
  3. testConstructWithFalse
  4. testCompareFalseAndSetSuccess
  5. testCompareFalseAndSetFailure

6. testCompareTrueAndSetSuccess
7. testCompareTrueAndSetFailure
8. testSetTrue
9. testSetFalse
10. testGetFalseAndSetTrue
11. testGetTrueAndSetFalse
12. testToStringTrue
13. testToStringFalse
14. testCompareFalseAndExchangeSuccess
15. testCompareFalseAndExchangeFailure
16. testCompareTrueAndExchangeSuccess
17. testCompareTrueAndExchangeFailure
18. testCompareFalseAndExchangeAcquireSuccess
19. testCompareFalseAndExchangeAcquireFailure
20. testCompareTrueAndExchangeAcquireSuccess
21. testCompareTrueAndExchangeAcquireFailure
22. testCompareFalseAndExchangeReleaseSuccess
23. testCompareFalseAndExchangeReleaseFailure
24. testCompareTrueAndExchangeReleaseSuccess
25. testCompareTrueAndExchangeReleaseFailure
26. testCompareAndSetAtomicity
27. testGetAndSetAtomicity
28. testCompareAndExchangeAtomicity
29. testCompareAndExchangeAcquireAtomicity
30. testCompareAndExchangeReleaseAtomicity

- Methods Tested:
  - Test 1-3 test the constructor
  - Test 4-7 test the method $compareAndSet$ considering equivalent classes
  - Test 8-9 test the method set
  - Test 10-11 test the method $getAndSet$ considering equivalent classes
  - Test 12-13 test the method $toString$ for both possible outcomes
  - Test 14-17 test the method $compareAndExchange$ considering equivalent classes
  - Test 18-21 test the method $compareAndExchangeAcquire$ considering equivalent classes
  - Test 22-25 test the method $compareAndExchangeRelease$ considering equivalent classes
  - Test 26-30 test the atomicity of all the update methods ($compareAndSet$, $getAndSet$, $compareAndExchange$, $compareAndExchangeAcquire$, $compareAndExchangeRelease$)

5.2 Testing AtomicInteger

5.2.1 Testing Base Class and Primitive

- Test Class: $BaseClassAndPrimitiveConcurrencyTest$
- Methods:
  - testIntegerRaceCondition
  - testPrimitiveIntRaceCondition
  - testAtomicIntegerNonRaceCondition

5.2.2 Testing Functionality and Atomicity

- Test Class: $AtomicIntegerTest$
- Test Methods:
  1. testDefaultConstructor
  2. testConstructorWithZero
  3. testConstructorWithPositive
  4. testConstructorWithNegative
  5. testConstructorWithMax
  6. testConstructorWithMin
  7. testConstructorWithPositiveExtreme
  8. testConstructorWithNegativeExtreme
  9. testSetZero
  10. testSetPositive
  11. testSetNegative
  12. testSetMax
  13. testSetMin
  14. testSetExtremePositive
  15. testSetExtremeNegative
  16. testGetAndSetZero
  17. testGetAndSetPositive
  18. testGetAndSetNegative
  19. testGetAndSetMax
  20. testGetAndSetMin
  21. testGetAndSetExtremePositive
  22. testGetAndSetExtremeNegative
  23. testCompareAndSetZero
  24. testCompareAndSetPositive
  25. testCompareAndSetNegative

26. testCompareAndSetMax
27. testCompareAndSetMin
28. testCompareAndSetExtremePositive
29. testCompareAndSetExtremeNegative
30. testCompareAndSetFailure
31. testGetAndIncrementZero
32. testGetAndIncrementPositive
33. testGetAndIncrementNegative
34. testGetAndIncrementMax
35. testGetAndIncrementMin
36. testGetAndDecrementZero
37. testGetAndDecrementPositive
38. testGetAndDecrementNegative
39. testGetAndDecrementMax
40. testGetAndDecrementMin
41. testGetAndAddZero
42. testGetAndAddPositive
43. testGetAndAddNegative
44. testGetAndAddMax
45. testGetAndAddMin
46. testGetAndAddMaxExtreme
47. testGetAndAddMinExtreme
48. testIncrementAndGetZero
49. testIncrementAndGetPositive
50. testIncrementAndGetNegative
51. testIncrementAndGetMax
52. testIncrementAndGetMin
53. testDecrementAndGetZero
54. testDecrementAndGetPositive
55. testDecrementAndGetNegative
56. testDecrementAndGetMax
57. testDecrementAndGetMin
58. testAddAndGetZero
59. testAddAndGetPositive
60. testAddAndGetNegative
61. testAddAndGetMax

62. testAddAndGetMin
63. testAddAndGetMaxExtreme
64. testAddAndGetMinExtreme
65. testCompareAndExchangeZero
66. testCompareAndExchangePositive
67. testCompareAndExchangeNegative
68. testCompareAndExchangeMax
69. testCompareAndExchangeMin
70. testCompareAndExchangeExtremePositive
71. testCompareAndExchangeExtremeNegative
72. testCompareAndExchangeFailure
73. testCompareAndExchangeAcquireZero
74. testCompareAndExchangeAcquirePositive
75. testCompareAndExchangeAcquireNegative
76. testCompareAndExchangeAcquireMax
77. testCompareAndExchangeAcquireMin
78. testCompareAndExchangeAcquireExtremePositive
79. testCompareAndExchangeAcquireExtremeNegative
80. testCompareAndExchangeAcquireFailure
81. testCompareAndExchangeReleaseZero
82. testCompareAndExchangeReleasePositive
83. testCompareAndExchangeReleaseNegative
84. testCompareAndExchangeReleaseMax
85. testCompareAndExchangeReleaseMin
86. testCompareAndExchangeReleaseExtremePositive
87. testCompareAndExchangeReleaseExtremeNegative
88. testCompareAndExchangeReleaseFailure
89. testGetAndUpdateAddition
90. testGetAndUpdateSubstraction
91. testGetAndUpdateMultiplication
92. testGetAndUpdateDivision
93. testGetAndUpdateNegation
94. testUpdateAndGetAddition
95. testUpdateAndGetSubtraction
96. testUpdateAndGetMultiplication
97. testUpdateAndGetDivision

98. testUpdateAndGetNegation
99. testGetAndAccumulateAddition
100. testGetAndAccumulateSubtraction
101. testGetAndAccumulateMultiplication
102. testGetAndAccumulateBitwiseAnd
103. testGetAndAccumulateBitwiseOr
104. testAccumulateAndGetAddition
105. testAccumulateAndGetSubtraction
106. testAccumulateAndGetMultiplication
107. testAccumulateAndGetBitwiseAnd
108. testAccumulateAndGetBitwiseOr
109. testGetAndSet
110. testCompareAndSet
111. testGetAndIncrement
112. testGetAndDecrement
113. testGetAndAdd
114. testIncrementAndGet
115. testDecrementAndGet
116. testAddAndGet
117. testGetAndUpdate
118. testUpdateAndGet
119. testGetAndAccumulate
120. testAccumulateAndGet
121. testCompareAndExchange
122. testCompareAndExchangeAcquire
123. testCompareAndExchangeRelease

- Methods Tested:
  - Test 1-8 test the constructor considering Robust Boundary Value testing
  - Test 9-15 test the method set considering Robust Boundary Value testing
  - Test 16-22 test the method $getAndSet$ considering Robust Boundary Value testing
  - Test 23-29 test the method $compareAndSet$ considering Robust Boundary Value testing
  - Test 31-35 test the method $getAndIncrement$ considering Normal Boundary Value testing
  - Test 36-40 test the method $getAndDecrement$ considering Normal Boundary Value testing
  - Test 41-47 test the method $getAndAdd$ considering Normal Boundary Value testing and Special Case Testing
  - Test 48-52 test the method $incrementAndGet$ considering Normal Boundary Value testing

- Test 53-57 test the method *decrementAndGet* considering Normal Boundary Value testing
- Test 58-64 test the method *addAndGet* considering Normal Boundary Value testing and Special Case Testing
- Test 65-72 test the method *compareAndExchange* considering Robust Boundary Value testing and Traditional Equivalent Class Testing
- Test 73-80 test the method *compareAndExchangeAcquire* considering Robust Boundary Value testing and Traditional Equivalent Class Testing
- Test 81-88 test the method *compareAndExchangeRelease* considering Robust Boundary Value testing and Traditional Equivalent Class Testing
- Test 89-93 test the method *getAndUpdate* considering Special Case Testing
- Test 94-98 test the method *updateAndGet* considering Special Case Testing
- Test 99-103 test the method *getAndAccumulate* considering Special Case Testing
- Test 104-108 test the method *accumulateAndGet* considering Special Case Testing
- Test 109-123 test the atomicity of all the update methods

6 Testing Outcome

6.1 AtomicBooleanTest

6.1.1 Stale Condition with Primitive and Base Class

First, I tested *Boolean* class and primitive *boolean* for thread safety. I used two threads for this, one to read from a shared boolean and another to write the same boolean. For a thread-safe behavior, upon flipping a boolean variable, the change should be visible to other threads. If the reader thread goes on indefinitely without being notified about the shared variable update, then the code is not thread-safe.

As I flipped a single boolean variable from *false* to *true* from the write thread, it was not visible to the reader thread in case of *Boolean* and *boolean*. The writer thread waited for reader thread to start and deliberately wasted some CPU cycles before flipping the shared variable value. But within a 5-second limit, only *AtomicBoolean* update was visible to reader thread. Increasing the timeframe may be sufficient for the reader thread to get the update and end on its own, but within this timeframe, only *AtomicBoolean* shows atomicity.

This demonstrates that base class and primitive value may be cached in the memory and threads may keep on working on an older value of shared variable, which may have unintended side effects.

One thing to note here is that I tried to demonstrate this with 999 threads sharing the same value stated together. My goal was to show that multiple threads observe the initial value and try to update it. But with the machine I am working with and with the JVM environment I have, the test was not successful. Rather, the test always showed that only one thread updated it, and the code behaves like all the threads saw the updates. It gave the wrong impression of thread safety.

```
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running util.concurrent.atomic.BaseClassAndPrimitiveConcurrencyTest
testAtomicBooleanNonStaleCondition
Reader exited after writer set sharedFlag. spins = 64472135
testBooleanStaleCondition
Reader still alive after writer set sharedFlag. spins = 0
testPrimitiveBooleanStaleCondition
Reader still alive after writer set sharedFlag. spins = 0
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 10.17 s -- in util.concurr
ent.atomic.BaseClassAndPrimitiveConcurrencyTest
```

Figure 1: Stale Condition with *Boolean* and *boolean*

These tests demonstrate the non-thread-safe vulnerability inherent in *Boolean* objects and *boolean* primitive, demonstrating the necessity of *AtomicBoolean*.

6.1.2 Functionality Test

I did functionality test for all the methods present in the *AtomicBoolean* class using Equivalent class testing. In this case, the input domain is only *false* and *true*. For methods involving *compare*, one more class is needed, which is the failed case of a comparison.

All the cases were passed, which implies the methods are correct. The correctness here is human-verified with method intention matching the output.

6.1.3 Atomicity with Atomic Class

I tested the update classes *compareAndSet*, *getAndSet*, *compareAndExchange*, *compareAndExchangeAcquire*, *compareAndExchangeRelease* for thread safety. All the tests were repeated 20 times.

The tests follow a similar pattern. I created 999 threads and started all the threads at the same time. The task was to switch *atomicBoolean* instance from false to true. I observed how many threads observe the initial false stage and change it to true. If the variable that is being flipped is atomic, then only one thread should see the false state, and the rest would see the updated true state.

All the tests passed with exactly one thread seeing the initial false flag, and the *atomicBoolean* was changed to true as expected. This demonstrated that the *AtomicBoolean* class is thread safe, unlike *Boolean* and *boolean*. This prevents duplicate execution, and when a flag is changed, it is visible to others.

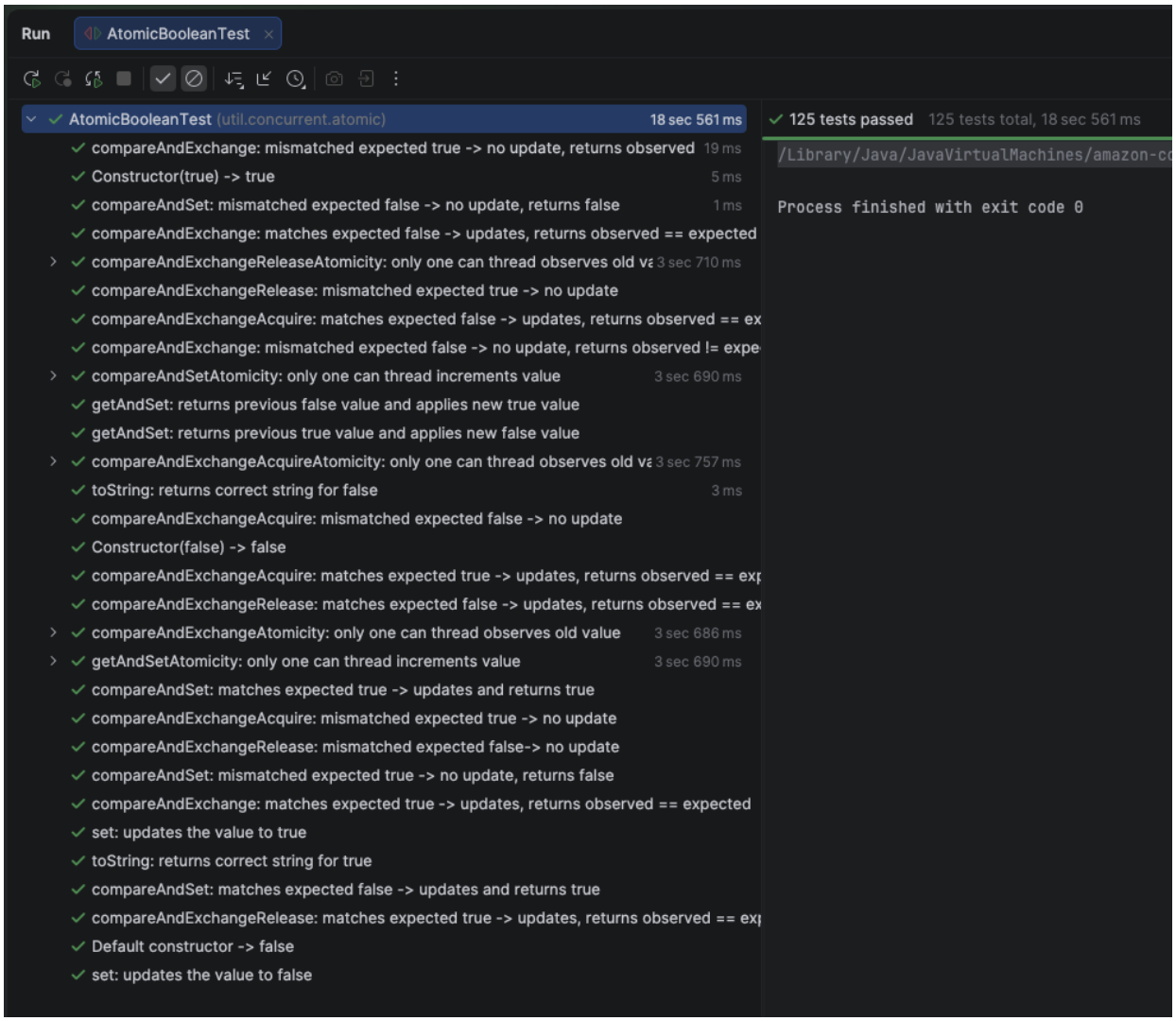


Figure 2: Atomic Boolean Code Concurrency Results

6.2 AtomicIntegerTest

6.2.1 Race Condition with Primitive and Base Class

First, I tested $Integer$ class and primitive int for race conditions with $AtomicInteger$. In each cases, I incremented a shared variable. The expected output here was the number of times each thread ran multiplied by the thread count, which was $THREAD_COUNT * INCREMENTS_PER_THREAD = 999 * 1000 = 999000$

The tests reveal that the results do not match the expected values. This confirms that there were multiple instances of threads accessing and overwriting the same variable at the same time, leading to missed and/or overwritten updates. In case of $AtomicInteger$, the expected result is matched. These tests demonstrate the non-thread-safe vulnerability inherent in $Integer$ objects and int primitive, demonstrating the necessity of $AtomicInteger$.

```
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running util.concurrent.atomic.BaseClassAndPrimitiveConcurrencyTest
testAtomicIntegerNonRaceCondition
Expected: 999000, Actual: 999000
testAtomicIntegerNonRaceCondition
Expected: 999000, Actual: 125075
testPrimitiveIntRaceCondition
Expected: 999000, Actual: 996616
[INFO] Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.595 s -- in util.concurr
ent.atomic.BaseClassAndPrimitiveConcurrencyTest
```

Figure 3: Race Condition with *Integer* and *int*

6.2.2 Functionality Test

I did functionality tests for all the methods present in the *AtomicInteger* classes using various testing methodologies. I used Robust Boundary Value Testing, Normal Boundary Value Testing, Special Case Testing, and Traditional Equivalent Class Testing strategies.

Robust Boundary Value Testing and Special Case Testing revealed the vulnerability associated with integer value overflow. The *AtomicInteger* class is prone to overflow problem where the value is more than $Integer.MAX_VALUE$ (2147483647) or less than $Integer.MIN_VALUE$ (−2147483648). It is the user's responsibility to use this class correctly. The IDE gives a warning if it can calculate that the value may overflow, but there are no compile-time errors for this.

Within the integer value boundary, all the methods behave as expected.

6.2.3 Atomicity with Atomic Class

I tested the update classes *getAndSet*, *compareAndSet*, *getAndIncrement*, *getAndDecrement*, *getAndAdd*, *incrementAndGet*, *decrementAndGet*, *addAndGet*, *getAndUpdate*, *updateAndGet*, *getAndAccumulate*, *accumulateAndGet*, *compareAndExchange*, *compareAndExchangeAcquire*, *compareAndExchangeRelease* methods for thread safety. These methods execute different types of update operations on *AtomicInteger* instances. All the tests were repeated 20 times.

For all the methods, I initialize an *AtomicInteger* with 0 and update it to 1 when a thread observes the value to be 0. If the access and update of the instance were not thread safe, then multiple threads would see the initial value of 0. In all cases, only one thread sees the initial value and updates it. This demonstrated that the *AtomicInteger* class is thread safe, unlike *Integer* and *int*. This prevents duplicate execution, and when a value is changes are immediately visible across threads.

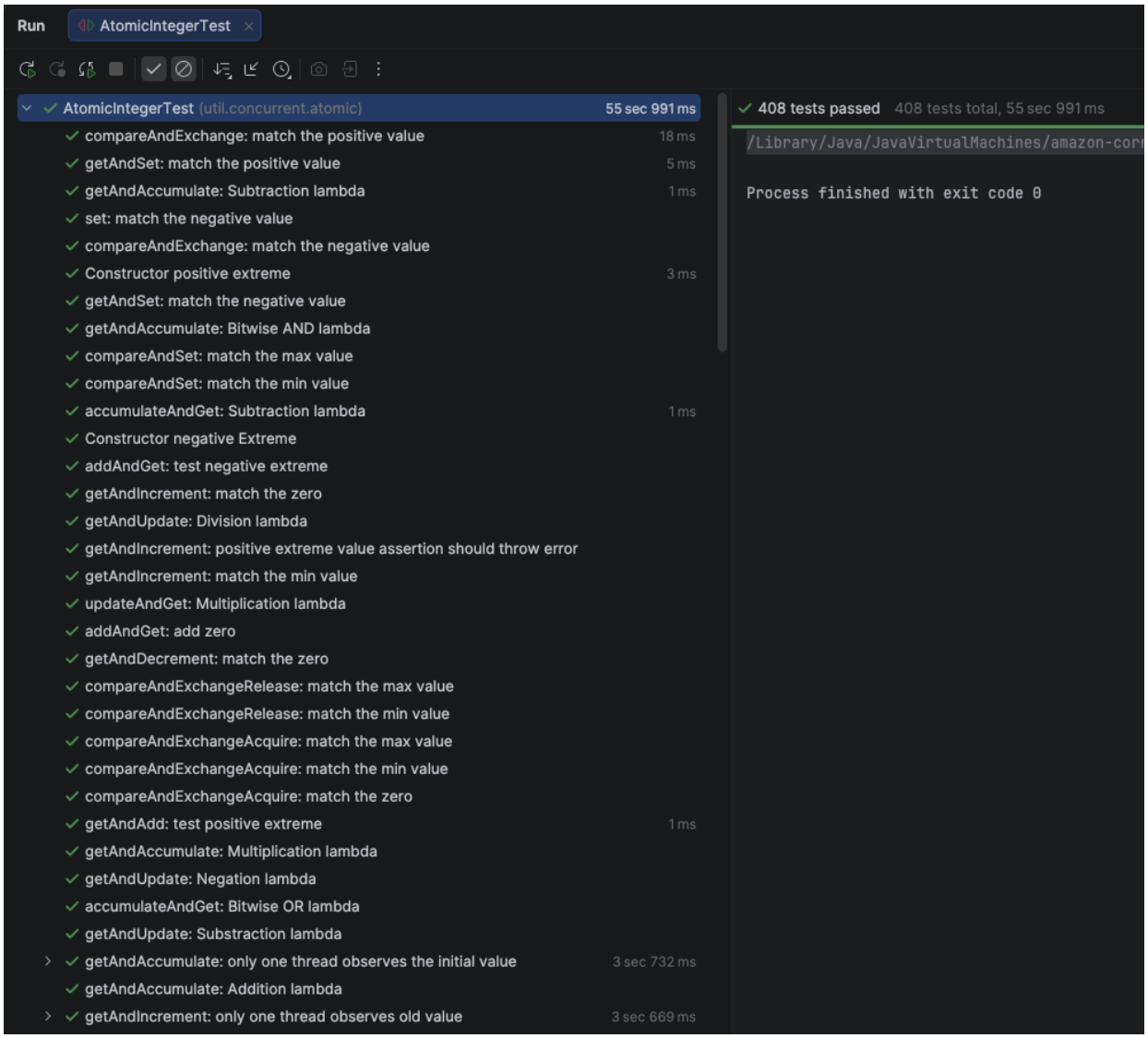
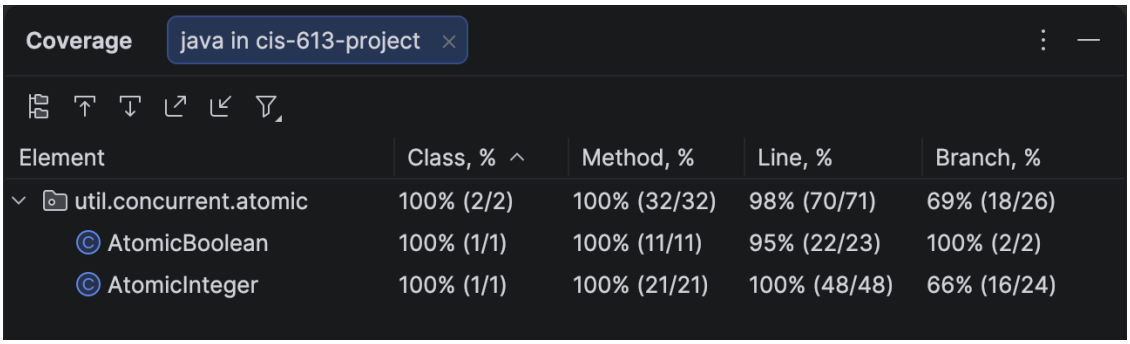


Figure 4: Atomic Integer Code Concurrency Results

7 Test Metrics

7.1 Test Coverage



Coverage | java in cis-613-project

Element	Class, %	Method, %	Line, %	Branch, %
util.concurrent.atomic	100% (2/2)	100% (32/32)	98% (70/71)	69% (18/26)
AtomicBoolean	100% (1/1)	100% (11/11)	95% (22/23)	100% (2/2)
AtomicInteger	100% (1/1)	100% (21/21)	100% (48/48)	66% (16/24)

Figure 5: Test Coverage for Classes Defined

The line that is not covered in *AtomicBoolean* class is a *catch* block. The branches not covered in *AtomicInteger* class are the ones related to unitary operation, where I did only special case testing.

7.2 Mutation Testing

The test suite obtained a mutation score of 98%. The survivor is related to *accumulateAndGet* methods condition negation, which is not the focus of correctness in this test suite.

Output Log

```
================================================================================
- Mutators
================================================================================
> org.pitest.mutationtest.engine.gregor.mutators.returns.PrimitiveReturnsMutator
>> Generated 15 Killed 15 (100%)
> KILLED 15 SURVIVED 0 TIMED_OUT 0 NON_VIABLE 0
> MEMORY_ERROR 0 NOT_STARTED 0 STARTED 0 RUN_ERROR 0
> NO_COVERAGE 0
--------------------------------------------------------------------------------
> org.pitest.mutationtest.engine.gregor.mutators.returns.BooleanTrueReturnValsMutator
>> Generated 8 Killed 8 (100%)
> KILLED 8 SURVIVED 0 TIMED_OUT 0 NON_VIABLE 0
> MEMORY_ERROR 0 NOT_STARTED 0 STARTED 0 RUN_ERROR 0
> NO_COVERAGE 0
--------------------------------------------------------------------------------
> org.pitest.mutationtest.engine.gregor.mutators.MathMutator
>> Generated 3 Killed 3 (100%)
> KILLED 3 SURVIVED 0 TIMED_OUT 0 NON_VIABLE 0
> MEMORY_ERROR 0 NOT_STARTED 0 STARTED 0 RUN_ERROR 0
> NO_COVERAGE 0
--------------------------------------------------------------------------------
> org.pitest.mutationtest.engine.gregor.mutators.returns.BooleanFalseReturnValsMutator
>> Generated 3 Killed 3 (100%)
> KILLED 2 SURVIVED 0 TIMED_OUT 1 NON_VIABLE 0
> MEMORY_ERROR 0 NOT_STARTED 0 STARTED 0 RUN_ERROR 0
> NO_COVERAGE 0
--------------------------------------------------------------------------------
```

```
> org.pitest.mutationtest.engine.gregor.mutators.returns.EmptyObjectReturnValsMutator
>> Generated 1 Killed 1 (100%)
> KILLED 1 SURVIVED 0 TIMED_OUT 0 NON_VIABLE 0
> MEMORY_ERROR 0 NOT_STARTED 0 STARTED 0 RUN_ERROR 0
> NO_COVERAGE 0
--------------------------------------------------------------------------------
> org.pitest.mutationtest.engine.gregor.mutators.NegateConditionalsMutator
>> Generated 28 Killed 28 (100%)
> KILLED 24 SURVIVED 0 TIMED_OUT 4 NON_VIABLE 0
> MEMORY_ERROR 0 NOT_STARTED 0 STARTED 0 RUN_ERROR 0
> NO_COVERAGE 0
--------------------------------------------------------------------------------
================================================================================
- Timings
================================================================================
> pre-scan for mutations : < 1 second
> scan classpath : < 1 second
> coverage and dependency analysis : 1 minutes and 10 seconds
> build mutation tests : < 1 second
> run mutation analysis : 7 minutes and 5 seconds
--------------------------------------------------------------------------------
> Total  : 8 minutes and 16 seconds
--------------------------------------------------------------------------------
> 1 tests took longer than 2000 ms
> Slowest test ([engine:junit-jupiter]/[class:util.concurrent.atomic.BaseClassAndPrimitiveConcurrencyTe
> Largest test ([engine:junit-jupiter]/[class:util.concurrent.atomic.AtomicBooleanTest]/[test-template:
================================================================================
- Statistics
================================================================================
>> Line Coverage (for mutated classes only): 78/80 (98%)
>> 5 tests examined
>> Generated 58 mutations Killed 58 (100%)
>> Mutations with no coverage 0. Test strength 100%
>> Ran 111 tests (1.91 tests per mutation)
Enhanced functionality available at https://www.arcmutate.com/
```

8 Model Checking

The domain I am writing the test suite for relates to model checking directly. As evident from the situations I faced with *Boolean* class and primitive, the behavior of the test is not deterministic at all times. Rather, it shows that the thing works for a particular setup. I used Java Path Finder (JPF) to test *AtomicBoolean* and *AtomicInteger* for model Checking.

Changes I had to make:

- Use Java 11.
- Lower the thread counts to much lower 3 for the tests instead of 999. Otherwise, the test eventually run out of memory.
- Test the Java library class directly, not the class I copied due to version conflict.

Within these constraints, JPF showed that the Java 11 *AtomicBoolean* and *AtomicInteger* are thread safe. Both the results returned no errors detected.

```
[→  cis-613-project git:(main) ✗ java -version
openjdk version "11.0.31" 2026-04-21 LTS
OpenJDK Runtime Environment Corretto-11.0.31.11.1 (build 11.0.31+11-LTS)
OpenJDK 64-Bit Server VM Corretto-11.0.31.11.1 (build 11.0.31+11-LTS, mixed mode)
[→  cis-613-project git:(main) ✗ javac AtomicBooleanJpfTest.java
[→  cis-613-project git:(main) ✗ java -jar /Users/kowshid/Documents/Repos/grad/jpf-core/build/RunJPF.jar check_atomic.jpf
JavaPathfinder core system v8.0 (rev 2bf98722c49a8149f9a6f68acb9edc6ba5cdd557) - (C) 2005-2014 United States Government. All rights reserved.


====================================================== system under test
AtomicBooleanJpfTest.main()

====================================================== search started: 4/22/26, 2:58 PM
[WARNING] orphan NativePeer method: jdk.internal.reflect.Reflection.getCallerClass(I)Ljava/lang/Class;
testAtomicBooleanNonStaleCondition
Reader exited after writer set sharedFlag. spins = 1
testAtomicBooleanNonStaleCondition
Reader exited after writer set sharedFlag. spins = 1
testAtomicBooleanNonStaleCondition
Reader exited after writer set sharedFlag. spins = 0
testAtomicBooleanNonStaleCondition
Reader exited after writer set sharedFlag. spins = 0
testAtomicBooleanNonStaleCondition
Reader exited after writer set sharedFlag. spins = 1
testAtomicBooleanNonStaleCondition
Reader exited after writer set sharedFlag. spins = 0

====================================================== results
no errors detected

====================================================== statistics
elapsed time:       00:00:00
states:             new=137,visited=139,backtracked=276,end=6
search:             maxDepth=18,constraints=0
choice generators:  thread=137 (signal=9,lock=24,sharedRef=35,threadApi=18,reschedule=51), data=0
heap:               new=798,released=204,maxLive=589,gcCycles=250
instructions:       16622
max memory:         256MB
loaded code:        classes=104,methods=2871

====================================================== search finished: 4/22/26, 2:58 PM
```

Figure 6: Model Checking for Atomic Boolean

```
[→  cis-613-project git:(main) ✗ java -version                                                                    ]
openjdk version "11.0.31" 2026-04-21 LTS
OpenJDK Runtime Environment Corretto-11.0.31.11.1 (build 11.0.31+11-LTS)
OpenJDK 64-Bit Server VM Corretto-11.0.31.11.1 (build 11.0.31+11-LTS, mixed mode)
[→  cis-613-project git:(main) ✗ javac AtomicIntegerJpfTest.java                                                  ]
[→  cis-613-project git:(main) ✗ java -jar /Users/kowshid/Documents/Repos/grad/jpf-core/build/RunJPF.jar check_atomic.jpf                ]
JavaPathfinder core system v8.0 (rev 2bf98722c49a8149f9a6f68acb9edc6ba5cdd557) - (C) 2005-2014 United States Government. All rights reser
ved.


====================================================== system under test
AtomicIntegerJpfTest.main()

====================================================== search started: 4/22/26, 2:59 PM
testAtomicIntegerNonRaceCondition
Expected: 9
Actual: 9
testAtomicIntegerNonRaceCondition
Expected: 9
Actual: 9
testAtomicIntegerNonRaceCondition
Expected: 9
Actual: 9
testAtomicIntegerNonRaceCondition
Expected: 9
Actual: 9

====================================================== results
no errors detected

====================================================== statistics
elapsed time:       00:00:00
states:             new=569,visited=1070,backtracked=1639,end=4
search:             maxDepth=22,constraints=0
choice generators:  thread=569 (signal=0,lock=9,sharedRef=110,threadApi=10,reschedule=440), data=0
heap:               new=1327,released=635,maxLive=532,gcCycles=1385
instructions:       59049
max memory:         256MB
loaded code:        classes=87,methods=2358
```

Figure 7: Model Checking for Atomic Integer

9 Discussion

This project taught me three major things.

- Java *concurrency* library is well built and suitable for projects requiring atomic operation on instances and variables.
- In regular code execution, it is not possible to say a race condition will definitely happen. Rather, most race conditions are probabilistic. I had to modify my code to achieve a race condition in case of boolean testing. This makes it really hard to figure out such corner cases of code and designs.
- For very large numbers, Java delegates the safe variable usage to the user. At the end, all variables use memory spaces, which are in bits. Monitoring for all the variable correctness at all times is not possible.

Appendices

A AI Usage

I used AI for these purposes:

- Figuring out how to synchronise multiple threads in Java
- Figuring out why $Boolean$ and $boolean$ were looked thread-safe in test methods for Race Condition with Primitive and Base Class 6.1.1.
- Figuring out how to use Java Path Finder for Model Checking in my project.